CHAPTER 17

MCP Clients in LangGraph

Why a brittle custom-tool approach breaks at scale, how the Model Context Protocol fixes it with a client/server split, and how to wire an async-only MCP adapter into a LangGraph chatbot that a synchronous Streamlit frontend can still call.

WHAT THIS CHAPTER COVERS

- The N × M tool-maintenance problem
- MCP as client/server separation of concerns
- Local (stdio) vs. remote (HTTP) MCP servers
- Why MCP adapters are async-only
- Bridging sync Streamlit with async MCP
- The persistent-adapter pattern (`mcp_manager.py`)
- Merging MCP tools with local tools in one graph
- Known limitations of the demo setup

MCP Clients in LangGraph

> **RECAP — CHAPTER 16**
>
> Chapter 16 connected a LangGraph chatbot to tools (`DuckDuckGoSearchRun`, a custom calculator, a custom `get_stock_price`) using `ToolNode`, `tools_condition`, and a `tools → chat_node` feedback edge so the graph could loop between reasoning and tool execution. Every one of those tools was hand-written: its request shape, authentication, and response parsing lived directly inside the chatbot's own codebase.

1. The Problem With Hand-Written Tools

Definition

A hand-written *tool* (a `@tool`-decorated function or a library wrapper such as `DuckDuckGoSearchRun`) packages one external capability directly into the chatbot's own source code. Every detail of how that capability talks to the outside world — request URL, headers, authentication, response parsing — lives inside the chatbot's repository.

Why It Matters: The Brittleness Problem

Consider a chatbot that needs to answer questions about an organization's GitHub repositories. Since no ready-made LangChain tool exists for this, the team writes a custom tool: it takes an owner, a repository URL, a pull-request state, and a result count; it attaches a GitHub token as an auth header; it calls `api.github.com/repos/.../pulls`; and it parses the JSON response into a human-readable list.

This works — until GitHub ships a breaking API change. A v1.0 → v2.0 upgrade might rename the endpoint from `/pulls` to `/pull_requests`, or rename response fields such as `title` to `title_name`. The moment that happens, the hand-written tool throws errors on every call, and a developer has to re-read GitHub's changelog and patch the tool by hand.

The Scaling Problem

One tool breaking occasionally is manageable. The real problem is scale:

- Connecting to one external service (GitHub) might require 5–10 separate custom tools (pull requests, commits, file counts, issues, and so on).
- A single company rarely has just one chatbot — it may run several chatbots across different teams, each independently wired to GitHub, Gmail, Slack, and other services.
- If there are *N* chatbots and each depends on *M* brittle custom tools, a single upstream API change can force updates across an **N × M** matrix of call sites.

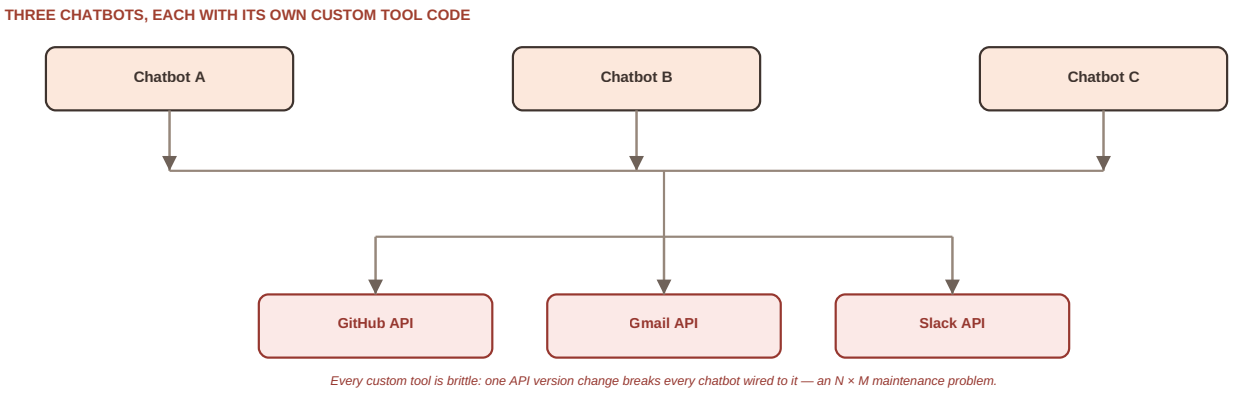


Figure 17.1 — Three chatbots, each maintaining its own brittle custom tool code against the same external APIs.

> **COMMON MISTAKE**
>
> Treating a hand-written tool as a one-time cost. The real cost is ongoing: every upstream API version bump becomes a maintenance ticket, multiplied by every chatbot that independently re-implemented the same integration.

> ↻ **Quick Revision — The Problem With Hand-Written Tools**
>
> - A hand-written tool hardcodes an external API's request/response details directly inside the chatbot.
> - An upstream API version change (e.g. GitHub v1.0 → v2.0) can silently break every tool built against the old shape.
> - With N chatbots and M custom tools each, a single breaking change can require fixes across an N × M matrix of call sites.
> - This is the core motivation for the Model Context Protocol (MCP).

2. What MCP Is and the Problem It Solves

Definition

MCP (Model Context Protocol) is a standardized way to connect tools to LLM applications. It is not a fundamentally new idea — it is an improved way of wiring up the same kind of tool access a chatbot already uses, built around one key architectural change: separating **where tool logic lives** from **where the chatbot lives**.

How It Works: Client/Server Separation

MCP splits the two parties in a tool integration into a **client** and a **server**:

- The **server** holds all of the actual tool implementation code — the part that previously lived inside the chatbot.
- The **client** — the chatbot side — holds only a small, stable piece of *configuration* that says which server to talk to and how to reach it.

Because the heavy, change-prone logic now lives entirely on the server, an upstream API change only requires updating the server. Every client that points at that server keeps working without a single code change on the client side, because the client never encoded any API-specific details in the first place.

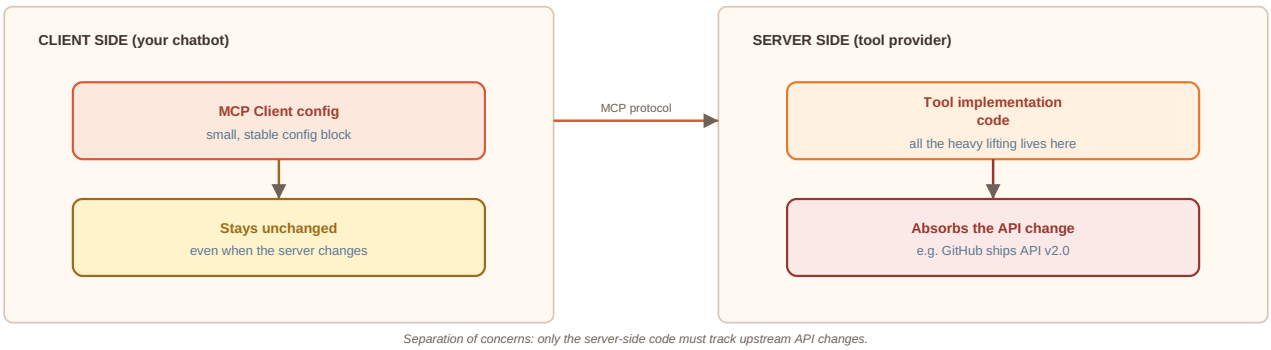


Figure 17.2 — MCP's client/server separation of concerns: the client carries only configuration; the server carries all the implementation detail.

IMPORTANT

MCP does not replace the idea of tools — it changes *where tool logic is implemented* and *how a chatbot discovers it*. An MCP server still ultimately exposes tools; the chatbot still ultimately calls them. What changes is that the chatbot no longer needs to know the implementation details in advance.

Local vs. Remote MCP Servers

Aspect	Local (stdio) server	Remote (HTTP) server
Where it runs	On the same machine as the client	Deployed separately, reachable over the network
Transport	Standard input/output (stdio)	Streamable HTTP
Client needs	A command to launch the server process	A URL and a transport setting
Typical use	Self-contained utilities (e.g. arithmetic)	Shared or third-party services (e.g. an expense tracker)

↻ Quick Revision — What MCP Is

- MCP is a standardized client/server protocol for connecting tools to LLM applications.
- Server: holds all implementation detail. Client: holds only a small, stable config block.
- An upstream API change only requires updating the server; client config stays untouched.
- Local servers use stdio transport; remote (deployed) servers use streamable HTTP.

3. Why the Chatbot Code Must Become Asynchronous

Problem It Solves / Why It Exists

Before an MCP client can be added to a LangGraph chatbot, the surrounding chatbot code generally has to become asynchronous, because the MCP integration libraries used to talk to MCP servers are async-only — they cannot be invoked through a plain synchronous call.

How It Works

Converting a LangGraph chatbot to async form means:

1. Any custom node function (e.g. a chat node) that calls the LLM must be declared `async def` and must call the LLM with `await llm_with_tools.ainvoke(...)` instead of the synchronous `.invoke(...)`.
2. The prebuilt `ToolNode` does not need manual conversion — it is already internally asynchronous.
3. The entry point of the script becomes an `async def main()` function, invoked through `asyncio.run(main())`, and any call into the compiled graph uses `await chatbot.ainvoke(...)`.

Why It Exists: The Benefit of Async Execution

Beyond being a requirement for MCP, asynchronous execution also allows independent pieces of work — such as two unrelated tool calls — to run concurrently rather than strictly one after another, which can reduce overall latency when a query needs more than one external lookup.

> **REAL WORLD INSIGHT**
>
> Asynchronous programming is its own large field of study. For this chapter, the only thing that matters is the narrow, practical reason it is introduced: the MCP client library requires it. A production chatbot will typically be async throughout for other reasons as well (concurrency, responsiveness), but that broader topic is out of scope here.

↻ Quick Revision — Why Async Is Required

- MCP adapter libraries only support asynchronous invocation — this forces the surrounding chatbot code to become async.
- Custom nodes become async def and call the LLM with await llm_with_tools.ainvoke(...).
- ToolNode is already async internally and needs no manual conversion.
- The script entry point becomes async def main(), run via asyncio.run(main()).

4. Building a Minimal MCP Client (Concept Walkthrough)

How It Works

At a conceptual level — independent of exactly which MCP library a given project ends up using — building an MCP client in LangGraph follows the same shape:

1. **Define server configuration.** For each MCP server the chatbot should reach, specify how to start or connect to it: a local server needs a launch command; a remote server needs a URL and transport.
2. **Create the client/adapter.** Instantiate the MCP client against that configuration.
3. **Discover tools.** Call the adapter's tool-listing method (e.g. `get_tools()` / `list_tools()`) to fetch the set of tools the server currently exposes, along with each tool's name, description, and parameters.
4. **Bind the discovered tools to the LLM** exactly as any other tool list would be bound, via `llm.bind_tools(tools)`.
5. **Wire the discovered tools into the existing `ToolNode`** so the chatbot's normal tool-calling loop (`chat_node → tools_condition → tools → chat_node`) can execute them.

> **REAL WORLD INSIGHT**
>
> The single biggest practical payoff of this pattern is that adding a brand-new capability — such as expense tracking against a remote MCP server — requires *zero* new application code on the client side. Pointing the same adapter at an additional server configuration is enough; the newly discovered tools flow into the existing bind/execute pipeline automatically.

> **BEST PRACTICE**
>
> A chatbot does not have to choose exclusively between hand-written tools and MCP tools. The two can coexist in the same `tools` list — a project can keep a couple of simple custom tools while delegating everything else to MCP servers. As a general direction, prefer MCP for anything likely to change upstream, since it is the more future-proof and robust integration path.

> ↻ **Quick Revision — Minimal MCP Client Pattern**
>
> • Steps: configure servers → create the client/adapter → discover tools → bind to the LLM → wire into ToolNode.
>
> • Adding a new MCP server requires no new client-side code — discovered tools flow straight into the existing pipeline.
>
> • Hand-written tools and MCP tools can coexist in the same tools list.

5. The Current MCP SDK V2 Implementation

Important Distinction

MCP tooling has moved fast. The specific client class used in this project's working code is the current `MCPAdapter` from `langchain.mcp`, not the older `MultiServerMCPClient` from `langchain_mcp_adapters.client`

that earlier tutorials (and this lecture) demonstrate. The underlying ideas — server config, tool discovery, async-only execution — carry over unchanged; only the specific import and class differ.

```
# mcp_manager.py
from langchain.mcp import MCPAdapter
```

> **COMMON MISTAKE**
>
> Installing the older `langchain-mcp-adapters` package alongside a current LangChain installation. The old package pins `mcp<2.0.0`, while the current `langchain[mcp]` dependency stack requires `mcp>=2.0.0` — the two cannot be installed together. The fix used in this project is to use `langchain.mcp.MCPAdapter` instead of reaching for the older package.

Server Configuration

The working configuration defines two MCP servers inside a single `SERVERS` dictionary: a local arithmetic server launched through `uv`, and a remote Zoho expense-tracking server reached through `npx mcp-remote` over HTTP. The Zoho server's API key is read from an environment variable rather than hardcoded.

```
# mcp_manager.py — server configuration
expense_api_key = os.getenv("ZOHO_API_KEY")

SERVERS = {
    "mcpServers": {

        # Local arithmetic MCP server, launched via uv
        "arith": {
            "command": "<PATH_TO_UV>/uv.exe",
            "args": ["run", "mcp", "run", "<PATH_TO_LOCAL_SERVER>/main.py"],
        },

        # Remote Zoho expense-tracking MCP server, via npx mcp-remote
        "expense": {
            "command": "npx",
            "args": [
                "mcp-remote",
                f"https://<ZOHO_MCP_HOST>/mcp/{expense_api_key}/message",
                "--transport", "http-only",
            ]
        }
    }
}
```

> **IMPORTANT**
>
> Real credentials and real hostnames are placeholders here (`<YOUR_API_KEY>`-style). Never commit or paste real API keys, tokens, or private server addresses into notes, chat logs, or source control.

↻ Quick Revision — Current MCP SDK V2

- Current project code uses langchain.mcp.MCPAdapter, not the older langchain_mcp_adapters.client.MultiServerMCPClient.
- The old adapter package pins mcp<2.0.0, which conflicts with current langchain[mcp] (mcp>=2.0.0) — this is why the switch was made.
- SERVERS config holds one local (uv-launched, stdio) server and one remote (npx mcp-remote, HTTP) server.
- Secrets (e.g. ZOHO_API_KEY) are read from environment variables, never hardcoded.

6. The Persistent-Adapter Hack: mcp_manager.py

Problem It Solves

A naive MCP integration would open the adapter, discover tools, and close the adapter again on every single request — wasteful, and awkward to reconcile with a synchronous frontend. Worse, the simplest async/await usage, if invoked directly from Streamlit, collides with the fact that Streamlit itself is fundamentally a synchronous library. The project's working code resolves this with a small dedicated module, `mcp_manager.py`, whose only job is to keep one MCP adapter alive for the whole process lifetime and provide a safe way for synchronous code to reach into it.

How It Works

1. A DEDICATED BACKGROUND EVENT LOOP

The manager creates its own `asyncio` event loop and runs it forever on a dedicated background thread, separate from whatever thread Streamlit itself is running on:

```
# mcp_manager.py
_ASYNC_LOOP = asyncio.new_event_loop()

_ASYNC_THREAD = threading.Thread(
    target=_ASYNC_LOOP.run_forever,
    daemon=True
)
_ASYNC_THREAD.start()


def run_async(coro):
    """Run a coroutine on the dedicated MCP event loop and wait for its result."""
    future = asyncio.run_coroutine_threadsafe(coro, _ASYNC_LOOP)
    return future.result()
```

`run_async()` is the bridge: any synchronous caller (such as a LangGraph node wrapper, see Section 7) can hand it a coroutine, and it submits that coroutine to the background loop with `asyncio.run_coroutine_threadsafe()`, then blocks until a result comes back.

2. STARTING THE ADAPTER ONCE AND KEEPING IT ALIVE

The adapter's async context manager is entered manually (`__aenter__`) rather than through a `with` block, specifically so it can stay open indefinitely instead of closing at the end of a single call. The coroutine then waits on an `asyncio.Event` that is never set, which is simply a way of parking the coroutine forever without returning — keeping the adapter's connection to both MCP servers alive for as long as the process runs:

```
# mcp_manager.py
_adapter = None
_mcp_tools = None

async def _start_mcp():
    global _adapter, _mcp_tools

    _adapter = MCPAdapter(SERVERS)
    await _adapter.__aenter__()

    _mcp_tools = await _adapter.list_tools()

    # Park here forever — this keeps the adapter's connections open
    await asyncio.Event().wait()
```

> **IMPORTANT**
>
> The `await asyncio.Event().wait()` line is not a real synchronization primitive being waited on by another part of the code — the event is created fresh and nothing ever calls `.set()` on it. Its only purpose is to suspend `_start_mcp()` indefinitely so that the `async with` context the adapter opened never exits. This is a deliberate, if unconventional, "stay alive forever" idiom.

3. TOOL-NAME COLLISION FIX

`MCPAdapter` (built on FastMCP) namespaces every discovered tool as `{server_name}_{tool_name}` — for example, `expense_add_expense`. In practice, the LLM provider used in this project (Groq, running an open-weight GPT-OSS model) tends to call tools by their original, un-prefixed MCP name (`add_expense`), and Groq's API rejects any tool call whose name was not present in the request's declared tool list. `mcp_manager.py` works around this by stripping the server prefix back off, but only when doing so is unambiguous:

```python
# mcp_manager.py
def _strip_server_prefix(tools):
    reserved = {"duckduckgo_search", "get_stock_price"}
    prefixes = tuple(f"{name}_" for name in SERVERS["mcpServers"])

    def stripped(name):
        for prefix in prefixes:
            if name.startswith(prefix):
                return name[len(prefix):]
        return name

    candidates = [stripped(t.name) for t in tools]

    for tool, new_name in zip(tools, candidates):
        if (new_name != tool.name
                and candidates.count(new_name) == 1
                and new_name not in reserved):
            tool.name = new_name

    return tools
```

COMMON MISTAKE

This fix only renames a tool when the un-prefixed name is unique across *all* tools (local and MCP) and does not collide with a reserved name. If two different MCP servers happened to expose a same-named tool, the prefix is deliberately left in place for both — silently renaming an ambiguous tool would risk routing a call to the wrong server.

4. EXPOSING TOOLS AND THE STARTUP GATE

`get_mcp_tools()` hands the already-discovered tool list to `backend.py`. `start_mcp()` submits `_start_mcp()` to the background loop and then busy-waits in short sleeps until `_mcp_tools` is populated, so that the rest of the module does not try to import tools before the adapter has actually connected:

```python
# mcp_manager.py
def start_mcp():
    future = asyncio.run_coroutine_threadsafe(_start_mcp(), _ASYNC_LOOP)
    while _mcp_tools is None:
        time.sleep(0.05)
    return future

def get_mcp_tools():
    if _mcp_tools is None:
        raise RuntimeError("MCPAdapter has not been started yet.")
    return _mcp_tools

# Start MCP as soon as this module is imported
start_mcp()
```

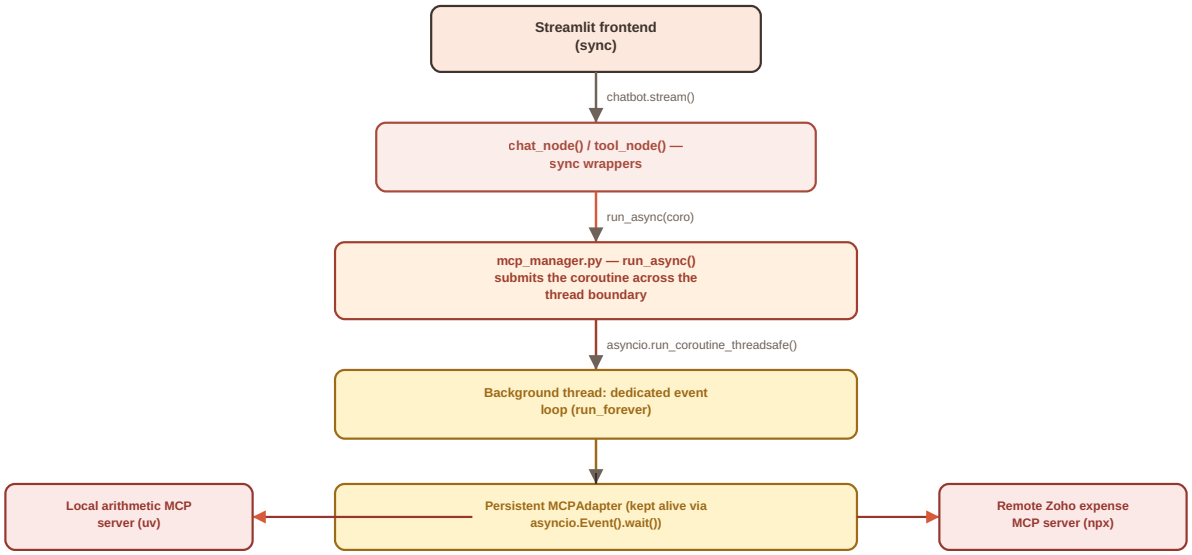


Figure 17.3 — How a synchronous Streamlit call reaches the persistent, async-only MCPAdapter through mcp_manager.py's background event loop.

↻ Quick Revision — mcp_manager.py

- A dedicated background thread runs its own asyncio event loop forever (run_forever).
- run_async(coro) bridges sync callers into that loop via run_coroutine_threadsafe(...).result().
- The adapter is entered manually (__aenter__) and kept alive by awaiting an asyncio.Event() that never fires.
- _strip_server_prefix() restores original tool names (e.g. add_expense) only when the un-prefixed name is unambiguous.
- start_mcp() runs once at import time and blocks (via short sleeps) until tools are actually discovered.

7. Wiring MCP Tools Into the LangGraph Chatbot

How It Works: backend.py

With `mcp_manager.py` handling the adapter lifecycle, `backend.py` only needs to import the already-discovered tools and merge them with any local tools before binding everything to the LLM:

```python
# backend.py
from mcp_manager import get_mcp_tools, run_async

mcp_tools = get_mcp_tools()

tools: list[BaseTool] = [
    search_tool,
    get_stock_price,
    *mcp_tools,
]

llm_with_tools = llm.bind_tools(tools)
```

Sync Wrappers Around Async Nodes

Streamlit's `chatbot.stream(...)` call expects LangGraph to behave synchronously, yet both the chat node and the tool node must do real work asynchronously (the LLM call, and any MCP tool execution). The pattern used is a thin synchronous wrapper around each async implementation, with `run_async()` doing the actual bridging:

```python
# backend.py
async def chat_node_async(state: ChatState):
    messages = state["messages"]
    response = await llm_with_tools.ainvoke(messages)
    return {"messages": [response]}

def chat_node(state: ChatState):
    """Synchronous wrapper: the real call still runs on the MCP event loop."""
    return run_async(chat_node_async(state))


tool_node_async = ToolNode(tools)

async def run_tool_node_async(state: ChatState):
    return await tool_node_async.ainvoke(state)

def tool_node(state: ChatState):
    return run_async(run_tool_node_async(state))
```

IMPORTANT

`run_async()` is a bridge between synchronous and asynchronous *code*, not a conversion of an async-only *tool* into a synchronous one. The MCP tool objects produced by `MCPAdapter` remain genuinely async — they still only support `.ainvoke()`. What `run_async()` does is let a synchronous caller (such as Streamlit, calling into the compiled graph) submit the coroutine that performs that async invocation to a loop that is already running, and then block until it finishes. The tool itself never became synchronous; the caller just stopped needing to manage the event loop by hand.

COMMON MISTAKE

Assuming that wrapping only the LLM node in a synchronous bridge is enough. If the graph's `ToolNode` is executed through LangGraph's plain synchronous `.invoke()` path instead of its async path, calling an MCP-generated tool (a `StructuredTool` that supports `.ainvoke()` only) raises `NotImplementedError: StructuredTool does not support sync invocation.` The fix is not to wrap the LLM node more carefully — it is to make sure the tool node itself runs through `ToolNode(...).ainvoke(state)`, as `run_tool_node_async()` does above.

Graph Topology Is Unchanged

Crucially, adding MCP tools does not change the shape of the graph at all — the same `chat_node → tools_condition → tools → chat_node` loop from Chapter 16 is reused unmodified. MCP tools and local tools sit side by side inside the same `tools` list and the same `ToolNode`:

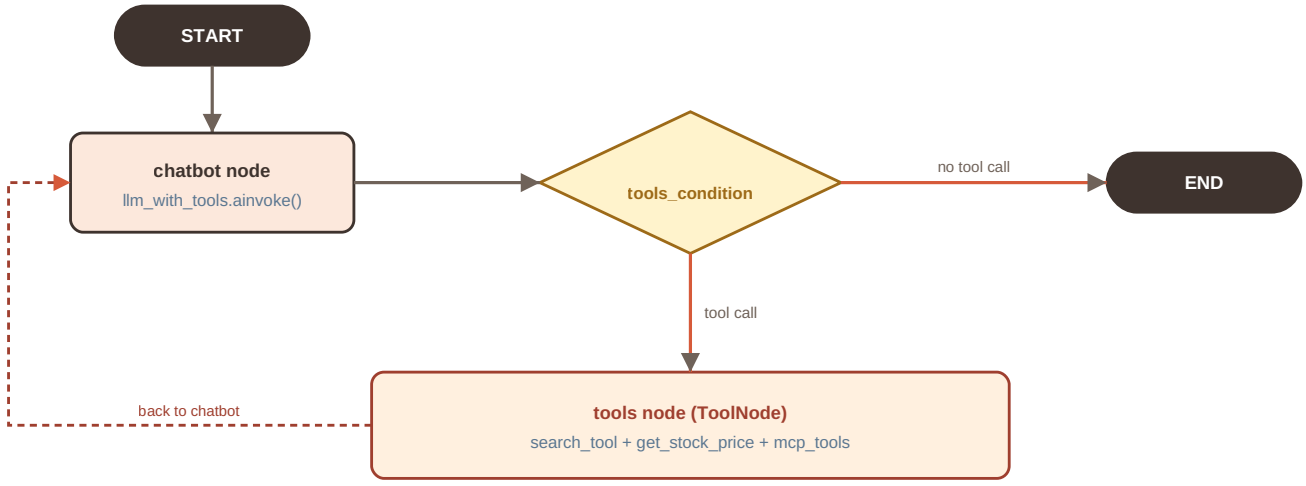


Figure 17.4 — The chatbot graph is unchanged from Chapter 16; MCP tools are simply additional entries inside the same ToolNode.

Checkpointing Stays Async Too

Because the rest of the graph is now running through the async bridge, the checkpointer is created with `AsyncSqliteSaver` over an `aiosqlite` connection rather than the synchronous `SqliteSaver` used in earlier chapters, and its setup call is itself awaited through `run_async()`:

```
# backend.py
async def init_checkpointer():
    conn = await aiosqlite.connect("chatbot.db")
    checkpointer = AsyncSqliteSaver(conn)
    await checkpointer.setup()
    return checkpointer

checkpointer = run_async(init_checkpointer())

chatbot = graph.compile(checkpointer=checkpointer)
```

↻ Quick Revision — Wiring MCP Into the Chatbot

- backend.py merges search_tool, get_stock_price, and *mcp_tools into one tools list before bind_tools().
- chat_node and tool_node are sync wrappers; the real async work is delegated via run_async().
- A sync ToolNode.invoke() cannot call an async-only MCP StructuredTool — the tool node must run through .ainvoke().
- The graph topology (chat_node / tools_condition / tools / feedback edge) is identical to Chapter 16.
- Checkpointing also moves to the async side: AsyncSqliteSaver over an aiosqlite connection.

8. Frontend: What Stays the Same

How It Works

`frontend.py` is left as a standard synchronous Streamlit app — it still calls `chatbot.stream({"messages": [...]}, config=CONFIG, stream_mode="messages")` exactly as in earlier chapters, and still distinguishes `ToolMessage` chunks (to update a status indicator) from `AIMessage` chunks (streamed as the visible reply).

This is only possible because every piece of asynchronous complexity was pushed down into `backend.py` and `mcp_manager.py`. From the frontend's point of view, `chatbot` is just another synchronous, streamable LangGraph object — it has no idea that two of its nodes are secretly bridging into a background event loop.

BEST PRACTICE

Keeping the frontend synchronous and pushing async complexity into the backend/manager layer is a deliberate isolation choice: it keeps the UI code simple and testable, and confines the trickiest part of the system (the sync/async bridge) to one well-understood module.

↻ Quick Revision — Frontend

- frontend.py is unchanged in shape: it still calls chatbot.stream(...) synchronously.
- All async/MCP complexity is isolated inside backend.py and mcp_manager.py.
- The frontend differentiates ToolMessage (status updates) from AIMessage (streamed reply text).

9. Known Limitations of This Implementation

What Is Confirmed Working

The adapter reliably starts and discovers its tools — a successful run prints a line such as:

```
MCPAdapter started. Loaded 6 tools.
```

This confirms that both the local arithmetic server and the remote Zoho expense server are reachable and that `MCPAdapter` can enumerate their tools. It does **not**, by itself, confirm that every discovered tool can be successfully called end-to-end through the full LangGraph loop.

Limitation 1 — The Adapter Must Stay Running Continuously

The "stay alive forever" pattern in `mcp_manager.py` (Section 6) only works for as long as the Python process itself keeps running. For a local demo — a single developer running `streamlit run frontend.py` in one terminal — this is a reasonable simplification: the process is already expected to run continuously for the length of the session.

> **REAL WORLD INSIGHT**
>
> This limitation is largely specific to ad-hoc local demos. In a real deployment, a backend service (a FastAPI app, a containerized worker, and so on) is already expected to run continuously as a long-lived process — which is exactly the condition this pattern needs. In that setting, keeping one adapter alive for the process lifetime is not a workaround at all; it is simply how a long-running service is supposed to behave. What is still missing, even for production use, is graceful startup-failure handling and a clean shutdown path for the adapter, since the current implementation has neither.

Limitation 2 — Tool-Call Validation Is Not Fully Verified

Groq's API validates that every tool name an LLM response calls was present in the `tools` list of that same request. The prefix-stripping fix in Section 6 addresses the common case (an MCP tool called by its un-prefixed name), but this project's testing has not yet exhaustively confirmed that every discovered MCP tool, across both servers, round-trips correctly through a live Groq tool call.

> **COMMON MISTAKE**
>
> Treating "the adapter started and logged 6 tools" as proof that the chatbot fully works end-to-end. Tool discovery and tool execution are two different steps in the pipeline — a tool can be discovered correctly and still fail at call time if its name, schema, or server-side implementation has an issue that only surfaces when the LLM actually tries to invoke it.

Limitation 3 — Three Libraries With Different Sync/Async Assumptions

The system has to reconcile three different concurrency models at once: MCP tools (async-only), LangGraph (comfortable with either sync or async), and Streamlit (fundamentally synchronous). The persistent background-loop pattern is the bridge that makes this combination work, but it is also the single most fragile part of the architecture — any future change to one of these three libraries' concurrency assumptions is the most likely place for this integration to break.

↻ Quick Revision — Limitations

- "Loaded N tools" confirms discovery, not that every tool executes correctly end-to-end.
- The adapter must be kept alive for the whole process lifetime — fine for a single continuously-running demo process or a real deployed service, not yet hardened with graceful shutdown or startup-error handling.
- Prefix-stripping fixes the common Groq tool-name mismatch, but full tool-call validation across every MCP tool is not yet exhaustively confirmed.
- The sync/async bridge across Streamlit, LangGraph, and MCP is the most fragile part of the current design.

K. Key Takeaways

- **The N × M problem:** hand-written tools hardcode external API details inside the chatbot, so one upstream API change can break every chatbot that depends on it.
- **MCP's core idea:** a standardized client/server split — the server holds all implementation logic, the client holds only small, stable configuration.
- **Async requirement:** MCP integration libraries are async-only, which forces the surrounding chatbot's nodes and entry point to become asynchronous.
- **Current SDK:** this project uses `langchain.mcp.MCPAdapter` (MCP SDK v2), not the older `MultiServerMCPClient`, due to a dependency version conflict.
- **mcp_manager.py:** a dedicated background event loop keeps one MCPAdapter alive for the process lifetime, with `run_async()` bridging synchronous callers into it.
- **Tool naming fix:** MCP-namespaced tool names (`server_tool`) are stripped back to their original name only when doing so is unambiguous, to satisfy Groq's tool-call validation.
- **Graph topology:** unchanged from Chapter 16 — MCP tools are simply additional entries in the same `ToolNode`.
- **Sync tool node is unsafe:** an async-only MCP `StructuredTool` cannot be called through a synchronous `ToolNode.invoke()`; it must run through `.ainvoke()`.
- **Current status:** tool discovery is confirmed working; full end-to-end tool-call correctness across every MCP tool is not yet exhaustively verified.

R. Revision Sheet

MCP

A standardized client/server protocol for connecting tools to LLM apps; server holds implementation, client holds config.

MCPAdapter

Current (SDK v2) LangChain MCP client class from `langchain.mcp`, replacing the older `MultiServerMCPClient`.

N × M Problem

N chatbots × M brittle custom tools each — one upstream API change can ripple across every call site.

Local vs. Remote Server

Local servers use stdio transport and a launch command; remote servers use a URL with streamable HTTP transport.

run_async()

Bridges synchronous callers into a persistent background asyncio event loop via `run_coroutine_threadsafe(...).result()`.

"Stay Alive Forever" Idiom

`await asyncio.Event().wait()` on an event that is never set, used to keep an adapter's context open indefinitely.

Tool Prefix Stripping

MCP tool names are namespaced as `server_tool`; stripped back to the original name only when unambiguous.

Sync/Async Tool Node Bug

A sync `ToolNode.invoke()` cannot call an async-only MCP tool — raises `NotImplementedError`; fix is to run the tool node via `.ainvoke()`.

I. Interview Questions

Q1. What specific problem does MCP solve that hand-written, per-chatbot tools do not?

Q2. Explain the client/server separation of concerns in MCP. Which side absorbs an upstream API change, and why?

Q3. Why must a LangGraph chatbot's code become asynchronous before an MCP client can be added to it?

Q4. What is the difference between a local (stdio) MCP server and a remote (HTTP) MCP server in terms of configuration?

Q5. Describe the role of `run_async()` in `mcp_manager.py`. Is it converting an async tool into a synchronous one? Why or why not?

Q6. Why does `mcp_manager.py` await an `asyncio.Event()` that is never set? What would happen if that line were removed?

Q7. Why does MCPAdapter namespace tool names as `server_tool`, and under what condition is it safe to strip that prefix back off?

Q8. A LangGraph chatbot throws `NotImplementedError: StructuredTool does not support sync invocation.` What is the likely root cause, and how is it fixed in this project's code?

Q9. Why does the checkpointer in this chapter use `AsyncSqliteSaver` over `aiosqlite` instead of the synchronous `SqliteSaver` used in earlier chapters?

Q10. What does "MCPAdapter started, loaded N tools" actually confirm, and what does it *not* confirm about the chatbot's correctness?

F. Further Reading

- Model Context Protocol — official specification and documentation (modelcontextprotocol.io).
- LangChain documentation — MCP integration guide, for the current adapter API surface.
- Python `asyncio` documentation — event loops, `run_coroutine_threadsafe`, and threading with asyncio.
- FastMCP documentation — for building and understanding MCP servers (server-side tool definitions, namespacing behavior).
- Streamlit documentation — notes on Streamlit's synchronous execution model and async-compatibility caveats.